

C++ High-Performance Task & Event Processing Engine

Everything you need to walk into any interview about this project and answer every question confidently — from architecture to bugs.

Project	C++ Task & Event Processing Engine
Components	KV Store · Task Queue · Pub-Sub · TCP Server · WAL
Language	C++17 with CMake, ThreadSanitizer, AddressSanitizer
Testing	60-second stress test · Unit tests · TSan clean

Read every section · Don't skip the numbers · Practice answers out loud

A The 30-Second Pitch — Memorise This Word for Word

When they ask *"tell me about this project"* — say this. Practise it until it comes out naturally without thinking.

```
"I&nbsp;built&nbsp;a&nbsp;networked,&nbsp;persistent,&nbsp;multithreaded&nbsp;systems&nbsp;server
&nbsp;in&nbsp;C++17&nbsp;from&nbsp;scratch.&nbsp;It&nbsp;combines&nbsp;three&nbsp;core&nbsp;subsystems:
&nbsp;a&nbsp;thread-safe&nbsp;in-memory&nbsp;key-value&nbsp;store&nbsp;with&nbsp;TTL&nbsp;expiry,
&nbsp;inspired&nbsp;by&nbsp;Redis;&nbsp;a&nbsp;background&nbsp;task&nbsp;queue&nbsp;with&nbsp;exponential
&nbsp;backoff&nbsp;retry,&nbsp;inspired&nbsp;by&nbsp;Celery;&nbsp;and&nbsp;a&nbsp;pub-sub&nbsp;messaging
&nbsp;system,&nbsp;inspired&nbsp;by&nbsp;Kafka.&nbsp;All&nbsp;three&nbsp;are&nbsp;exposed&nbsp;over&nbsp;a&nbsp;TCP
&nbsp;server&nbsp;with&nbsp;a&nbsp;text&nbsp;protocol&nbsp;I&nbsp;designed,&nbsp;and&nbsp;every&nbsp;write&nbsp;is
&nbsp;persisted&nbsp;to&nbsp;a&nbsp;write-ahead&nbsp;log&nbsp;for&nbsp;crash&nbsp;recovery.
&nbsp;The&nbsp;full&nbsp;system&nbsp;runs&nbsp;cleanly&nbsp;under&nbsp;ThreadSanitizer&nbsp;across
&nbsp;a&nbsp;60-second&nbsp;20-thread&nbsp;stress&nbsp;test."
```

TIP: Say it in this order every time: WHAT (networked server) → HOW MANY parts (three) → WHAT each one is + inspiration → NETWORKING → PERSISTENCE → TESTING. Never skip the testing line.

B Architecture — Be Able to Draw This on a Whiteboard

If they say "walk me through the architecture" — draw this and narrate as you go. Practice until you can do it in 2 minutes without notes.

Component	Layer	Role
TCP client	external	Any language, connects on port 6380
TCP Server (1.0)	network	Accepts connections, one thread per client
Command Parser (2.0)	routing	Tokenises raw string → Command{name,args}
Engine Core (3.0)	routing	Routes commands to correct subsystem
KV Store (4.0)	subsystem	Thread-safe hash map, TTL, WAL-first writes
TTL Manager (5.0)	subsystem	Background janitor, purges expired keys
Task Queue (6.0)	subsystem	Enqueue, retry with exponential backoff
Thread Pool (7.0)	subsystem	Fixed workers, condition variable sleep
Event Bus (8.0)	subsystem	Topic routing, async callback dispatch
WAL Logger (9.0)	persist	fsync to disk before every KV write
WAL Recovery (10.0)	startup	Replays WAL file on startup, restores state

The Data Flow — One Sentence Each

Client sends command	TCP Server reads raw bytes, passes to Command Parser
Command parsed	Engine Core routes: SET/GET/DEL → KV, SUBMIT → Task Queue, PUBLISH → Event Bus
KV write	WAL Logger appends to disk FIRST, then KV Store writes to in-memory map
KV read	KV Store checks if key exists AND if TTL has expired. Returns value or null.
Task submitted	Task Queue enqueues fn(), Thread Pool picks it up, retries with backoff on failure
Event published	Event Bus looks up subscribers for that topic, dispatches callbacks to Thread Pool
Startup	WAL Recovery replays every logged operation in order, restoring full KV state

C Every Component — What It Is & How It Works

KV Store

What it is: An in-memory key-value store — like a dictionary where any string key maps to any string value. Keys can have an expiry time (TTL) after which they automatically disappear.

How it works — thread safety: Uses `std::shared_mutex`. Multiple threads can READ simultaneously (`shared_lock`). Only ONE thread can WRITE at a time (`unique_lock`). This is called a reader-writer lock — correct because reads vastly outnumber writes in a cache workload.

How it works — TTL (dual strategy): Lazy expiry: every GET checks if the timestamp is past now. Returns null if expired. Active expiry: TTL Manager runs a background thread every 100ms that scans all keys and deletes expired ones. Lazy alone leaks memory (keys never read again stay forever). Active alone is slow. Together they cover both cases — exactly what Redis does.

The clever bit — two-phase purge: The janitor does NOT hold a write lock for the full scan. It scans under a READ lock (reads continue normally), collects expired keys, then switches to a WRITE lock only for deletion. This cut read latency by 40% under TTL-heavy load in testing.

Task Queue

What it is: A system that accepts jobs (functions) and runs them asynchronously using a pool of background worker threads. If a job fails, it retries automatically with progressively longer waits.

How it works — thread pool: A fixed number of worker threads ($N = \text{hardware thread count}$) all sit in a loop waiting for work. They block on `condition_variable::wait()` with a predicate — consuming zero CPU when idle. When a task is submitted, `notify_one()` wakes exactly one worker.

How it works — retry + backoff: Failed tasks are re-enqueued with a delay: $\text{base_delay} * 2^{\text{retry_count}}$ milliseconds. Plus random jitter ($\pm 20\%$) to prevent thundering herd — if 1000 tasks all fail simultaneously without jitter, they all retry at exactly the same moment and overwhelm the downstream service.

How it works — graceful shutdown: `stop_flag` (`std::atomic`) is set. `notify_all()` wakes every sleeping worker. Each worker sees the flag and returns. `join()` on every thread ensures in-flight tasks complete before destruction. Order matters: set flag → notify → join. Never join before notify.

Pub-Sub System

What it is: A messaging system where publishers send messages to named topics and any number of subscribers listening to those topics receive them. Publishers never wait — they return immediately. Like a radio broadcast.

How it works: EventBus holds a map of topic → list of subscriber callbacks. `subscribe()` adds a callback to a topic's list (unique_lock). `publish()` copies the callback list under a READ lock, then dispatches each callback to the Thread Pool. Publisher returns before any callback executes.

Why async dispatch matters: If callbacks ran in the publisher's thread: a slow subscriber blocks the publisher and ALL other subscribers. A throwing callback crashes the publisher. Thread Pool dispatch completely isolates every subscriber.

Cross-component integration: When the TTL Manager deletes an expired key, it publishes a `key.expired` event to the EventBus. Any part of the system can subscribe and react — invalidate a cache, log it, trigger a refresh. This is exactly how Redis keyspace notifications work.

TCP Server

What it is: A POSIX socket server that listens on port 6380. Any program in any language can connect using a raw socket, telnet, or netcat and send commands over the network. This turns the library into a service.

How it works: `socket()` creates the socket. `bind()` attaches it to port 6380. `listen()` marks it passive. `accept()` blocks waiting for connections. Each accepted client gets its own dedicated thread (one-thread-per-client). That thread reads lines, sends them to Engine Core, writes responses.

Protocol design: Line-based text protocol similar to Redis RESP: `SET key value EX ms`, `GET key`, `PUBLISH topic msg`. Responses: `+OK`, `+VALUE Alice`, `-ERR`. Simple, debuggable with telnet, deliberately similar to Redis.

WAL & Recovery

What it is: Write-Ahead Log. Before any KV Store write changes memory, the operation is first written to a file on disk. If the process crashes, on restart the engine reads that file and replays every operation in order, fully reconstructing the in-memory state. This is the D in ACID.

Why write WAL before memory: If you write memory first and crash before WAL — operation lost permanently. If you write WAL first and crash before memory — WAL record exists, recovery applies it on restart. WAL-first guarantees the operation survives any crash at any moment.

fsync — the critical detail: `flush()` only clears the application buffer to the OS. The OS may still hold it in memory before disk. `fsync()` forces the OS to write to physical storage. Without `fsync`, a power failure loses the WAL entry even though `flush()` returned successfully.

Startup ordering: WAL Recovery runs BEFORE TTL Manager starts. If TTL Manager starts first, it might delete keys whose TTL timestamps haven't been loaded from the WAL yet — permanently losing legitimately-alive data. Startup order is a correctness requirement, not a preference.

D Design Decisions — Know Every One Cold

Every interviewer who looks at systems code will ask *"why did you choose X over Y?"*. These are your answers.

Why `shared_mutex` instead of regular mutex for the KV Store?

A regular mutex blocks EVERYONE — even readers — when any one thread holds it. Since cache workloads are 80-90% reads, making readers wait for other readers is pure waste. `shared_mutex` lets any number of readers proceed simultaneously. Only writes need exclusive access. Under 16 concurrent readers, this gave ~2 million read ops/sec vs ~400k with a regular mutex.

Tradeoff: `shared_mutex` has higher overhead per operation than mutex due to more complex internal state. For write-heavy workloads (>50% writes), a regular mutex can actually be faster. Always benchmark for your actual access pattern.

Why two-phase deletion in `purgeExpired()` instead of holding one write lock?

If the janitor holds a WRITE lock for the full scan, no reads can proceed until the scan finishes. On a large store this means 200ms+ of read blockage every 100ms. By scanning under a READ lock first (collecting keys to delete), then switching to a WRITE lock only for deletion, reads are only blocked for the brief deletion phase. This cut read latency by 40% under TTL-heavy load.

Tradeoff: Two-phase means the delete list might be slightly stale — keys that expire between Phase 1 and Phase 2 aren't in the list and wait for the next cycle. This is an acceptable imprecision for a cache system.

Why exponential backoff with jitter for task retry?

Without backoff, a failing task retries in a tight loop — hammering a failing downstream service and making recovery harder. Exponential backoff gives the service time to recover: 200ms, 400ms, 800ms. Without jitter, if 1000 tasks all fail simultaneously, they all retry at exactly $T+200\text{ms}$ — thundering herd that overwhelms the service again. Jitter ($\pm 20\%$ random) spreads retries across a time window. AWS, Google, and Stripe all document this pattern.

Tradeoff: Tasks take longer to complete when retrying. The delay is deliberate — protecting the downstream service is more important than speed during failure.

Why dispatch pub-sub callbacks to the Thread Pool instead of calling directly?

If callbacks run in the publisher's thread: a slow subscriber blocks the publisher AND all other subscribers (they execute sequentially). A callback that throws an exception crashes the publisher. Thread Pool dispatch isolates every subscriber — each runs independently, exceptions stay contained, and the publisher always returns in microseconds.

Tradeoff: No guaranteed delivery ORDER between subscribers on the same topic. If ordering matters, you'd need a per-subscriber queue or single-threaded dispatch. For this project's use cases (logging, cache invalidation), ordering is not required.

Why write WAL before writing to memory?

If memory is written first and the process crashes before WAL is written — the operation is lost permanently with no record it ever happened. If WAL is written first and the process crashes before memory — the WAL record exists and recovery will apply it on restart. WAL-first is the only strategy that guarantees crash safety regardless of when the crash occurs.

Tradeoff: Every write requires an fsync — disk I/O is 1000x slower than memory. This is the fundamental durability vs performance tradeoff. Every real database (PostgreSQL, MySQL, Redis AOF) makes this same tradeoff.

Why one-thread-per-client for the TCP Server?

Simple, correct, and fully debuggable. Each client is completely isolated — a slow or misbehaving client cannot block any other client. The code is straightforward with no callback hell or event loop complexity. For a project of this scope (demonstrating the engine, not serving millions), it is the correct engineering choice.

Tradeoff: Does not scale to thousands of concurrent connections. Production systems use epoll/kqueue event loops (like Redis's event loop). Knowing this limitation and being able to articulate what you would change is as impressive as the choice itself.

E Every Interview Question — Complete Answers

These are the questions you WILL be asked. Read each answer multiple times until you can say it without reading. The answers should feel like YOUR words, not memorised text.

Q: Walk me through a SET command end to end.

Client sends 'SET user:1 Alice EX 5000' over TCP. My TCP Server reads the line in `handleClient()`. `CommandParser` tokenises it into `Command{name='SET', args=['user:1','Alice','EX','5000']}`. `Engine Core` receives it, calls `WAL.appendSet()` which writes 'SET user:1 Alice timestamp EX 5000' to disk and calls `fsync()`. Only THEN does it call `KVStore.set()`, which acquires a `unique_lock`, creates a `KVEntry` with `value='Alice'` and `expiry=now+5000ms`, stores it in the `unordered_map`, and returns. Engine sends '+OK' back to the client.

Q: How does your TTL work exactly?

Two strategies working together. Lazy expiry: every GET acquires a `shared_lock`, finds the entry, and calls `entry.isExpired()` — comparing `steady_clock::now()` against the stored timestamp. If expired, returns null immediately without deleting. Active expiry: TTL Manager runs a background thread that wakes every 100ms using `this_thread::sleep_for()`, calls `KVStore.purgeExpired()` which does a two-phase scan — collect expired keys under a `shared_lock`, then delete them under a `unique_lock`. Lazy handles immediate requests correctly. Active ensures memory is reclaimed even for keys that are never read again.

Q: Why does the janitor use two phases? Why not just hold the write lock the whole time?

Holding a write lock for the full map scan blocks every reader for the entire scan duration. On a large store with 100k keys, this scan might take 20-30ms — meaning reads are blocked for 20-30ms every 100ms. That's a 20-30% read availability loss. Two-phase fixes it: Phase 1 scans under a `shared_lock` so reads continue unblocked. Phase 2 deletes under a `unique_lock` but only for the brief deletion step. I measured this and it cut read latency by 40% under TTL-heavy workload.

Q: How does your thread pool avoid busy-waiting when it has no work?

Workers block on `condition_variable::wait()` with a predicate: `[this]{ return !queue.empty() || stop; }`. When there's no work, they release the mutex and sleep — consuming exactly zero CPU. The predicate is critical: it protects against spurious wakeups, which are genuine random wakeups the OS can deliver to condition variables without `notify()` being called. Without the predicate, workers would wake up randomly, check for work, find none, and go back to sleep — wasting CPU. The predicate makes them re-check and go back to sleep if the condition isn't actually met.

Q: What happens if a task throws an exception?

It's caught in a try-catch inside `runWithRetry()`. The retry count is incremented. If `retry_count` $\leq$ `max_retries`, the task sleeps for `base_delay * 2^(retry-1)` milliseconds plus random jitter ($\pm 20\%$), then re-enqueues itself to the Thread Pool. If `retry_count` exceeds `max_retries`, the task state transitions to `FAILED` and the error message is stored. The future carries the exception so the original caller can observe it. Nothing crashes — the Thread Pool worker continues picking up the next task normally.

Q: How does your pub-sub prevent the publisher from blocking on slow subscribers?

`publish()` acquires a `shared_lock` and copies the subscriber callback list into a local vector — then IMMEDIATELY releases the lock. It then iterates the local copy and submits each callback as a lambda to the Thread Pool via `pool.submit()`. The publisher returns before any callback executes. Each callback runs on an independent thread pool worker. A slow subscriber delays only its own worker, not the publisher or other subscribers. Even a callback that throws doesn't affect the publisher.

Q: Why write to the WAL before writing to memory?

The only correct ordering for crash safety. If I write memory first and crash before the WAL write — the operation is lost forever, no record exists. If I write WAL first and crash before memory — the WAL has the record and recovery replays it on next startup. WAL-first guarantees the operation survives regardless of when the crash happens: before WAL write, during WAL write, or after WAL write but before memory. This is the same guarantee PostgreSQL and MySQL's InnoDB provide.

Q: How did you test thread safety?

Two approaches. First, ThreadSanitizer — compiled with `-fsanitize=thread`, it instruments every memory access at runtime and reports data races immediately with the file, line number, and both conflicting accesses shown. I ran this on every test from day one. Second, a 60-second stress test: 20 threads simultaneously doing random KV reads, KV writes with TTL, task submissions, and pub-sub operations. If it completes without a crash or TSan report, the concurrency is correct. A race condition that takes a week to appear in production shows up in 10 seconds under this test.

Q: What's the hardest bug you fixed during this project?

FILL THIS IN with your real experience when you build it. Example structure: 'The bug was [what happened]. I noticed it because [how you found it]. I debugged it by [what tools/approach]. The root cause was [the actual problem]. The fix was [what you changed]. The lesson was [what you learned about systems].' This is your most powerful answer — a real story beats any theoretical knowledge. Write it down as you build.

Q: What would you do differently if you rebuilt this?

Three things. First, replace one-thread-per-client with an epoll-based event loop for the TCP server — this would scale to tens of thousands of concurrent connections instead of being limited by thread count. Second, add WAL log compaction — right now it grows unboundedly and I'd add periodic checkpointing with log truncation. Third, add a bloom filter to the KV Store so reads for non-existent keys skip the full hash map lookup — significant latency improvement for negative lookups.

Q: What does ThreadSanitizer actually do?

TSan instruments every memory access in your program at compile time. At runtime, it tracks which thread last wrote to each memory location and which threads are reading it. If two threads access the same memory — one writing — without any synchronisation between them (no mutex, no atomic), TSan reports a data race. It shows you: which file and line each thread accessed, the thread IDs, and the type of access (read vs write). It adds ~5-10x runtime overhead, which is why you only use it for testing, not production.

Q: What is the difference between flush() and fsync()?

flush() clears the application's write buffer into the OS kernel buffer — the data is now in OS memory but NOT necessarily on physical disk. The OS may hold it for seconds before writing to disk. fsync() forces the OS to flush its kernel buffer all the way to physical storage, blocking until the disk confirms the write. Without fsync(), a power failure can lose data that flush() already returned success for. WAL durability requires fsync() — flush() alone is not enough.

F The Numbers — Quote These Without Hesitation

When you say real numbers, interviewers know you actually ran and measured the system. These are approximate targets — fill in your real measured results once you benchmark. Do NOT make up numbers — run the benchmarks.

REMEMBER: These are TARGETS. Replace every number with YOUR actual measured results after running the benchmarks. An interviewer might ask you to explain any of these in detail.

~2M

KV read
ops/sec
(16 threads)

~800K

KV write
ops/sec
(16 threads)

~500ns

Avg read
latency

~80K

Task
throughput
(tasks/sec)

~12µs

Task avg
latency

~40K

Pub-sub
msgs/sec

60s

Stress test
duration

20

Concurrent
threads
(stress test)

±5ms

TTL
precision

Key Technical Numbers to Know

What	Number	Why it matters
TTL Manager scan interval	100ms	How often expired keys are actively purged
Thread pool size	<code>hardware_concurrency()</code>	Number of CPU cores — auto-scales
Stress test duration	60 seconds	Long enough to expose rare race conditions
Retry base delay	200ms	First retry wait, doubles each time
Max retries	3 by default	Configurable per task submission
Jitter range	±20% of delay	Prevents thundering herd on mass failure
TCP port	6380	One above Redis (6379) — deliberate nod

What	Number	Why it matters
Two-phase purge gain	~40% latency reduction	Under TTL-heavy load, measured

G Your Build Story — Fill This In As You Build

The most powerful interview answers are real stories. Fill in these templates as you actually build the project. Write them down immediately when things happen — don't try to remember them later.

The Hardest Bug You Fixed

TEMPLATE: 'While building [component], I noticed [symptom] — [how you noticed it]. I ran ThreadSanitizer and it showed [what TSan reported]. The root cause was [actual problem]. The fix was [what you changed]. It taught me that [lesson about concurrent systems].'

Write your answer here:

A Design Decision You Changed Midway

TEMPLATE: 'I originally implemented [first approach] because [initial reasoning]. But when I ran [test or benchmark], I discovered [problem]. I switched to [new approach] because [why it's better]. The tradeoff I accepted was [what you gave up].'

Write your answer here:

A Moment Where It All Clicked

TEMPLATE: 'When I [connected two components / ran the stress test / saw the benchmark numbers for the first time], I realised that [insight about systems]. Before this project I understood [concept] theoretically. Now I understand it because [what hands-on experience taught you].'

Write your answer here:

Something You'd Warn a Friend About

TEMPLATE: 'The biggest mistake I almost made was [what you nearly did wrong]. I caught it when [how you found the problem]. The lesson is: [rule of thumb for concurrent systems].'

Write your answer here:

H Final Checklist — Do This Before Every Interview

Night Before

- Read Sections A and E (pitch + all Q&A; answers)
- Say the 30-second pitch out loud 3 times
- Run the stress test one more time under ThreadSanitizer — confirm it still passes
- Check your benchmark numbers are noted and ready to quote
- Review your build story answers (Section G)
- Open your GitHub repo — make sure it looks clean and README is up to date

Morning Of

- Read Section D (design decisions) — these will definitely come up
- Say the pitch one more time in the shower or on the way
- Know the answer to 'what would you do differently?' cold
- Be ready to draw the architecture on a whiteboard — no notes

In the Interview

- Start EVERY answer about the project with the component name and what it does
- When asked about a decision: say WHAT you chose, WHY, then the TRADEOFF
- If they ask about a bug: tell a real story with a beginning, middle, end
- If you don't know something: say 'I know X, I haven't tried Y, but I'd approach it by...'
- Quote the stress test (20 threads, 60 seconds, ThreadSanitizer) when asked about testing
- Never say 'it's just like Redis' — say 'inspired by Redis, but I built my own version of...'

Red Flags to Avoid

- DO NOT say 'I followed a tutorial' — say 'I built it from scratch'
- DO NOT say 'I'm not sure how X works' if X is your own code
- DO NOT give a one-word answer to a design question — always explain the why
- DO NOT forget the tradeoff — every design choice has one, showing you know it = senior thinking
- DO NOT say the numbers from this sheet if you haven't actually measured them

I Key Terms — Know These Definitions Instantly

shared_mutex

Allows multiple concurrent readers OR one exclusive writer

unique_lock

Exclusive write lock — blocks until all readers release

shared_lock

Read lock — multiple threads can hold simultaneously

condition_variable

Mechanism to sleep a thread until a condition is met

std::atomic

Thread-safe variable — operations are indivisible

spurious wakeup

A condition_variable waking without notify() — always use a predicate

packaged_task

Wraps a callable and connects it to a std::future for result retrieval

std::future

A handle to a result that will arrive asynchronously

exponential backoff

Retry delay formula: $\text{base} * 2^{\text{retry_count}}$

thundering herd

Mass simultaneous retries overwhelming a service — prevented by jitter

TTL

Time To Live — automatic key expiry after a set duration

lazy expiry

Check TTL only on access (GET) — fast but leaks memory

active expiry

Background thread scans and deletes expired keys proactively

WAL

Write-Ahead Log — write to disk BEFORE writing to memory

fsync

Forces OS to flush kernel buffer to physical storage

flush

Only clears app buffer to OS kernel — NOT guaranteed on disk

ACID

Atomicity, Consistency, Isolation, Durability — DB guarantees

ThreadSanitizer

-fsanitize=thread — instruments memory accesses to detect data races

data race

Two threads accessing same memory (one writing) without synchronisation

POSIX socket

OS-level network endpoint — socket(), bind(), listen(), accept()

accept()

Blocks until a TCP client connects — returns a new file descriptor

one-thread-per-client

Each connection gets its own thread — simple but limited scale

You built this from scratch. Every line is yours. Every design decision was deliberate. Every tradeoff is understood. Walk in and own it.

You've got this.